

多阶段多目标优化下的农作物种植策略

摘要

充分利用有限的耕地资源，因地制宜地采取种植策略，同时提升经济效益对于乡村经济的可持续发展具有重要意义。本文针对以上问题，根据某乡村 2023 年的种植规划、农作物信息与地块 (含大棚) 分布等数据，基于**多阶段多目标优化**、**随机模拟**等研究方法，对于农作物的种植策略进行分析，并采取 **0-1 整数规划**、**遗传算法**对于模型进行求解。

针对问题一，首先进行数据分类汇总，获取各个农作物的销售数据、各类地块 (含大棚) 的面积分布。之后，基于问题中给出的多项种植规则分析得出不同地块之间存在的**独立性**与依赖关系，将问题转化为**多阶段优化问题**，并编制能够反映农作物种植分散程度与经济效益的目标函数，从而对于每个阶段分别进行**多目标优化**。针对豆类作物三年一种的问题，本文将问题转化为构建张量，采用 **0-1 整数规划**求解。最后，采取**轮植**的方法避免连茬问题，便于实施管理。

针对问题二，本文首先**建立各农作物各项参数变化的概率模型**，依据多数农作物的预期销售量较稳定变化这一特点，提出在问题一求解的基础上对于每一年各类农作物的种植面积**浮动调整**，通过局部区块地互补调整提高经济效益，避免大幅变动种植面积分布，同时满足不连茬与豆类作物三年一种条件，并采取**遗传算法**解决多阶段的优化问题。

针对问题三，本文通过查找文献确定农作物的替代与互补关系、销售量与销售价格之间的相互作用。将全体农作物划分为 3 个替代类，在问题二求解的基础上，采取**随机模拟**的方法，在替代类中随机扰动各个作物的种植面积，据此得出该乡村 2024 ~ 2030 农作物的最优种植方案。

关键字： 多阶段多目标优化 0-1 整数规划 随机模拟 遗传算法 轮植

一、问题重述

1.1 问题背景

农业对于乡村经济具有重大意义，由于耕地资源、作物需求量有限，为提高经济效益，满足可持续发展等要求，需要优化当地农作物的种植策略。

本文需对一个包括多个大小与类型各异的地块与大棚的乡村进行农作物种植面积规划。并同时同时考虑作物适应性、作物替代性与互补性、重茬、豆类作物种植、密度与面积等限制条件，以及市场需求、种植成本、预期销售量与销售价格的波动。

因此，需要综合考虑各项约束条件与市场供需情况，分析各作物种类之间的关系，权衡生产效益与种植风险，建立适当的数学模型对种植策略进行布局、评估与优化，以期制定出最优的种植策略。

1.2 题目(含附件)信息

- 从 2023 年起所有地块(含大棚)必须三年一种豆类作物；
- 作物不能连茬种植；
- 过剩的作物有两种可能的对待方式：滞销或者折价出售；
- 附件中给出耕地、农作物基本信息、2023 年度农作物种植与销售情况、种植季对应月份。

1.3 待求解问题

以下三个问题具有递进关系，均以多个约束条件、现实因素下的作物种植决策为主题，并从一般性的假设出发，逐步考虑更复杂的情况：

问题一：分析附件 1 和附件 2 的数据，在农作物的预期销售量、种植成本、亩产量和销售价格保持稳定的假设下，分别针对过剩滞销和过剩折价出售两种情况，确定该乡村 2024 至 2030 年作物种植方案，以最大化经济效益。

问题二：考虑不同作物预期销售量、亩产量、种植成本与销售价格的波动，给出该乡村 2024 至 2030 年的农作物种植方案。

问题三：在问题二的基础上，考虑农作物之间存在的可替代性和互补性，以及预期销售量与销售价格、种植成本之间的相关性，制定该乡村 2024 至 2030 年农作物的种植方案，通过模拟数据进行求解，将结果与问题二中的结果进行比较分析。

二、问题分析

2.1 问题一的分析

问题一：研究对象为各种作物的种植面积分布，附件 2 中只提供了各个地块上的作物种植面积，因此需要先对于该数据进行预处理，以获取各种作物的种植面积，从而估计其预期销售量。首先对原始数据进行异常值、缺失值检测。之后需要对于附件 2 表 1 和附件 2 表 2 的数据进行匹配，以计算得出每种作物的预期销售量。

各个地块之间的种植关系并不平等，有些地块并不互相干涉。首先按照三年一种豆类作物的限制，确定豆类作物在 2024 ~ 2030 年之间在哪些地块上种植。其次一部分地块的种植面积与其他地块完全不重合，例如 D^2, E^2 ，可以**分离为单独的决策优化问题考虑**。进一步进行地块划分，则可以分为粮食类地块 A, B, C 和蔬菜类地块 D^1, E, F 考虑。下图给出作物与地块之间的关联。在相应种植策略下橙色矩形区域内的作物的种植与其他作物**独立**。

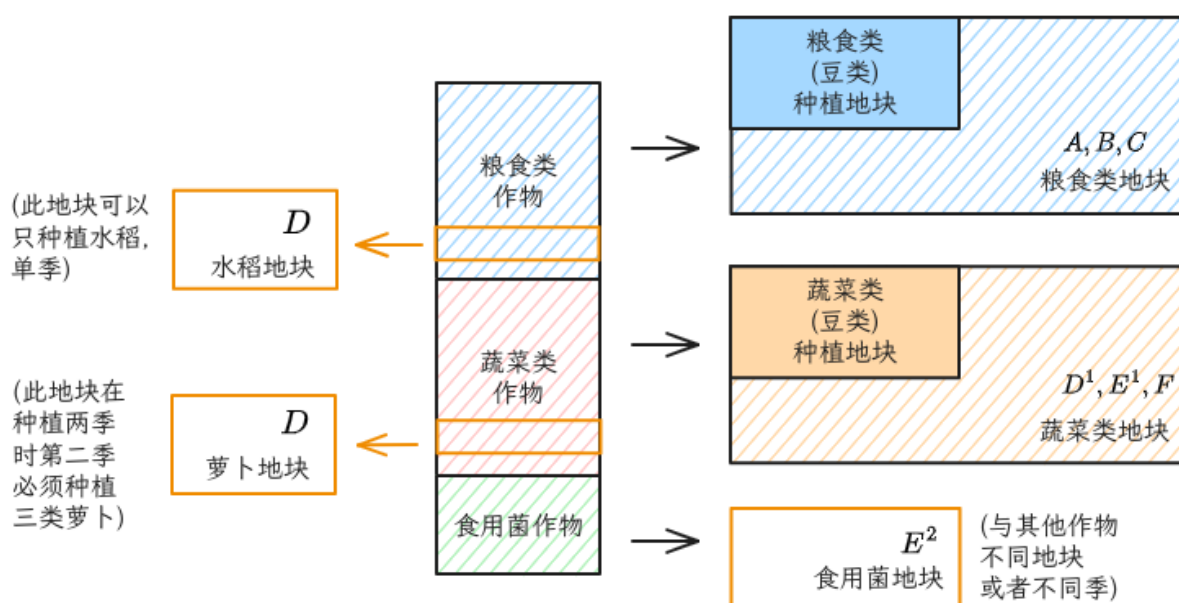


图 1 作物与地块之间的所属关系

根据不同种植方案、过剩销售情况下的作物依赖关系，建立不同的递进的**线性规划**问题，其中豆类作物的种植可以通过 **0-1 整数规划**求解。

2.2 问题二的分析

问题二的求解可以部分地依赖于问题一的分析。主要分为两步进行：首先**建立**各个农作物未来的预期销售量、种植成本、亩产量、销售价格变化的**概率模型**；第二步，根据题干中多数农作物的预期销售量变化稳定 (在 $-5\%, 5\%$ 之间浮动) 这一信息，可以在

第一问求解的结果上对于每一年各类农作物的种植面积进行**互补调整**，即限制在同一地块上种植的作物不变，通过调整种植面积获取最大的经济效益。

2.3 问题三的分析

问题三的求解必须建立在问题二的基础上，首先通过查找文献确定销售价格与销售量之间关系，以及作物之间的可替代关系与互补关系。之后可以采取随机模拟的方法确定作物的种植策略。但同时为了保证不连茬以及其他约束条件，可以通过在问题二当年的种植面积分布上进行修正得到解。因此将地块分为若干类，分别进行动态调整。

三、模型假设

1. 假设数据集中存在的属性噪声和标签噪声在可接受范围内，不会掩盖特征与目标变量之间可能存在的关系；
2. 假设数据集中所给出的基因中存在能够对目标变量产生显著性影响的基因；
3. 农作物均当季销售或者滞销，不存在不同时间种植的农作物的干扰关系。

四、符号说明

符号	说明
A, B, C, D, E, F	平旱地、梯田、山坡地、水浇地、普通大棚、智慧大棚地块类型
$A_i, B_i, C_i, D_i, E_i, F_i$	相应地块类型第 i 个地块
$D_i^k, E_i^k, F_i^k (k = 1, 2)$	相应地块类型第 i 个地块第 k 季
$S_{A_i}, S_{B_i}, S_{C_i}, S_{D_i}, S_{E_i}, S_{F_i}$	第 i 个 $A(B, C, D, E, F)$ 地块的面积
Fd, Veg, Fu	粮食类 (非豆类)、蔬菜类 (非豆类)、食用菌
Fd^d, Veg^d	粮食类 (豆类)、蔬菜类 (豆类)
$Crop_i^*$	作物类型为 $Crop(Fd, Veg, Fu, Fd^d, Veg^d)$ 的第 i 个作物
$M_{i,Z}^t$	种植在 Z 地块上的作物 i 第 t 年的预期销售量 (斤)
$C_{i,Z}^t$	种植在 Z 地块上的作物 i 第 t 年的种植成本 (元/亩)
$Y_{i,Z}^t$	种植在 Z 地块上的作物 i 第 t 年的亩产量 (斤/亩)
$P_{i,Z}^t$	种植在 Z 地块上的作物 i 第 t 年的销售价格 (元/斤)

五、数据侧写

附件一与附件二中给出了各个地块与作物的信息，经过整理如下：

作物类型	类数	种植地块	说明
Fd_i^d	5	A, B, C	A, B, C 都是单季
$Fd_i(i = 1, \dots, 10)$	10	A, B, C	
Fd_{11}	1	D	水稻, D 适宜每年种植一季水稻或者两季 Veg
Veg_i^d	3	D^1, E^1, F	F 表示 F_1, F_2 , 下同
$Veg_i(i = 1, \dots, 15)$	15	D^1, E^1, F	
$Veg_i(i = 16, 17, 18)$	3	D^2	大白菜、白萝卜、红萝卜; 只能由 D^2 种植
Fu_i	4	E^2	

各个作物在不同类型地块上的种植面积如下：

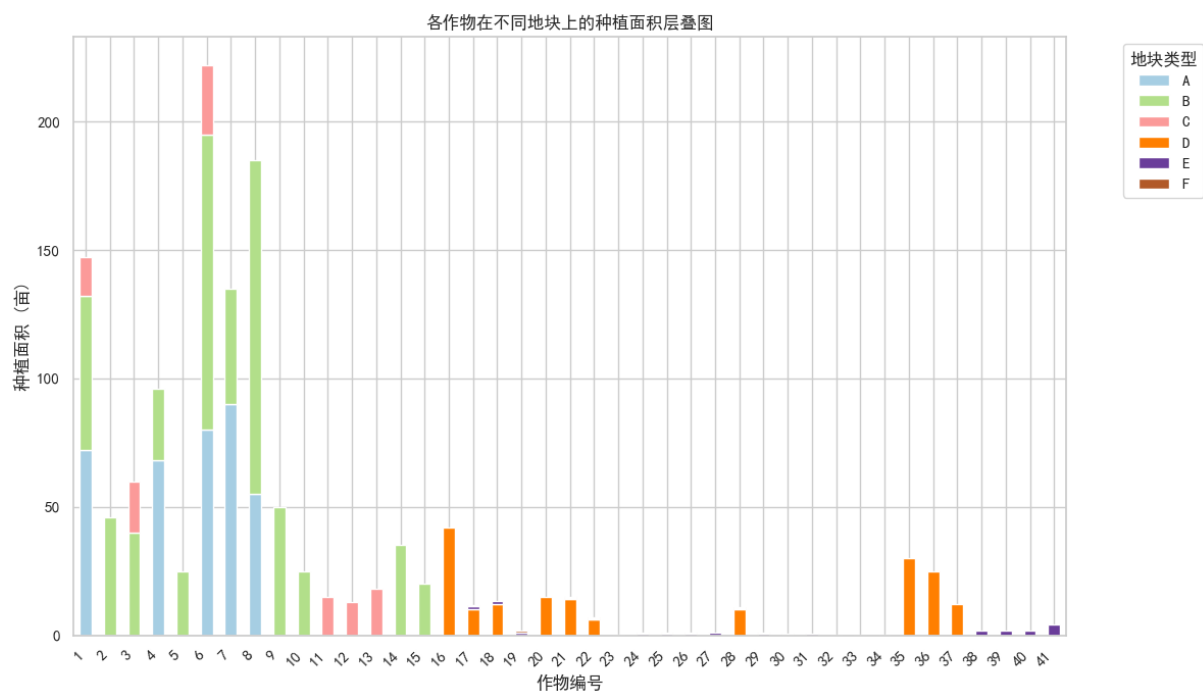


图2 各个作物在不同地块上的种植面积

其中豆类作物的编号为: $1, \dots, 5$ (Fd^d) 和 $17, 18, 19$ (Veg^d). 此种植面积分布反映了如下的关键信息，对于之后的模型建立过程可能产生影响：

1. 豆类作物 (粮食类、蔬菜类 (除 19 以外)) 的需求量在相应的地块中种植面积较大，说明其预期销售量较高；

2. 绝大部分蔬菜作物 (23 ~ 27, 29 ~ 34) 的需求量极小;
3. *D* 地块上构成竞争关系的作物水稻 (16) 与萝卜类 (35, 36, 37) 的种植面积相持 (42 亩与 67 亩, 具体数据见附录), 两者均具有较高的需求量。

各地块 (含大棚) 面积分布如下:

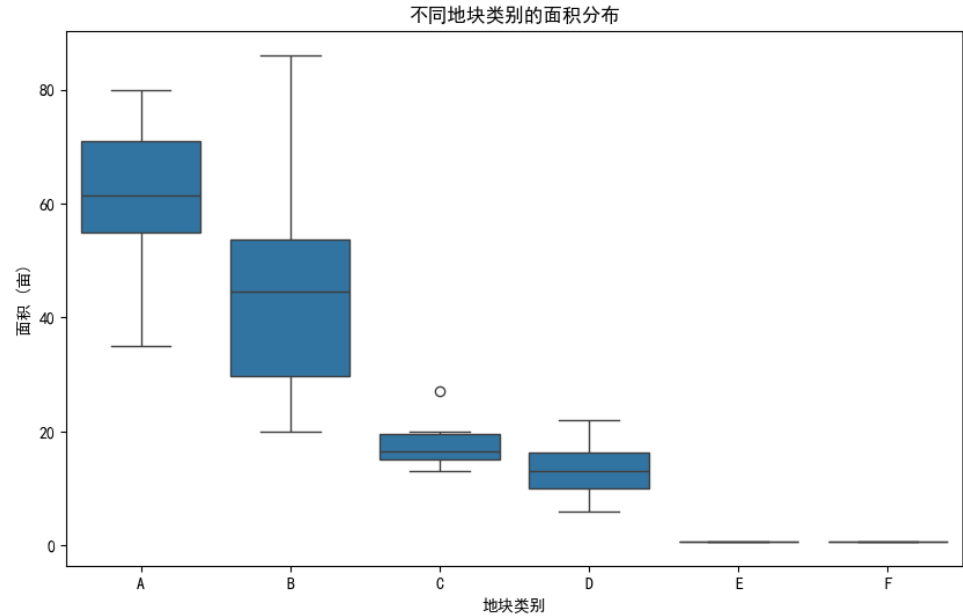


图3 各个地块 (含大棚) 的种植面积分布

地块类型	总地块面积	地块个数	地块类型	总地块面积	地块个数
A	365.0	6	D	109.0	8
B	619.0	14	E	9.6	16
C	108.0	6	F	2.4	4

六、问题一模型建立与求解

问题一的求解主要分为如下步骤进行:

- 首先运行豆类作物种植方案搜索模型, 确定豆类作物在各地块 (含大棚) 中应当如何种植;
- 计算各个地块剩余的种植面积 S_Z , 作为后续约束优化;
- 确定各个地块之间的依赖关系, 对于独立的地块进行单独优化;
- 对于剩余的区块进行多目标决策优化;
- 采取轮植的方法进行作物种植。

6.1 豆类作物的种植策略

根据题目中豆类作物必须三年一种，以及不能连茬的要求，刻画数学模型如下：

构建张量 $Q \in \mathbb{R}^{I \times T \times S}$. $Q = (q_{its})$ 其中 $q_{its} \in \{0, 1\}$ 表示作物 i 在第 t 年是否在地块 s 上种植. 根据 i) 不能连茬种植和 ii) 每一地块三年内必须种植一次豆类植物, 张量首先应当满足如下条件: 其每一行的相邻 3 个元素的和应当不小于 1:

$$\sum_{t=t}^{t+2} q_{*^d ts} \geq 1, \forall 1 \leq t \leq 6, \forall 1 \leq s \leq S \quad (1)$$

$$\sum_{t=t}^{t+1} q_{*^d ts} \leq 1, \forall 1 \leq t \leq 7, \forall 1 \leq s \leq S \quad (2)$$

其中 $*^d$ 表示 Fd^d 或者 Veg^d 。

说明：根据题干中的描述, A, B, C 无需考虑多季, D, E 均只有第一季可以种植 Veg^d ; F 虽然有多季都可以种植 Veg^d , 但其总地块面积仅有 2.4 亩, 并不能很好地满足豆类作物种植面积要求, 同时考虑到简化操作要求, 仅使用 F^1 种植豆类作物。因此, 在豆类作物种植方案分配环节不考虑第 2 季的豆类作物种植。

同时, 所选取的地块面积不能太低, 否则将无法种植豆类作物, 已经求解出豆类作物所需的 $S_{\max}$ 和 $S_{\min}$, 引入 $\beta = [b_1, \dots, b_S]^T$ 表示地块面积 (注: 以不同季区分地块), 数据见附件。应当超过最小需求面积并且小于最大需求面积:

$$\sum S_{*^d, \min} \leq \sum_{s=1}^S q_{*^d ts} \beta_s \leq \sum S_{*^d, \max}, \forall 2 \leq t \leq 8 \quad (3)$$

对于 A, B, C 地块进行张量求解, 以最大化利润, 参数设置如下:

参数	数值	参数	数值
I	5	S	26
T	8	$*^d$	Fd^d

对于 D^1, E^1, F 进行张量求解, 以最大化利润, 参数设置如下:

参数	数值	参数	数值
I	3	S	28
T	8	$*^d$	Veg^d

6.2 过剩滞销

6.2.1 D 地块上只种植水稻时的约束规划模型

这种情况下 D 只种植水稻, 实际上只需要考虑 A, B, C, E, F 需要解决如下的优化问题:

假设 Q 为可选的豆类种植方案 (下同), 对于 $Q_0 \in \mathcal{O}$, X_{Q_0} 为此豆类作物种植方案获取到的利润, S_Z 将由 Q_0 决定。

$$\begin{aligned}
 \max \quad & X_{Q_0} + \sum_{i=1}^{10} \sum_{Z=A,B,C} S_{Fd_i,Z} (Y_{Fd_i,Z} \cdot P_{Fd_i,Z} - C_{Fd_i,Z}) \\
 & + \sum_{i=1}^{15} \sum_{Z=E^1,F} S_{Veg_i,Z} (Y_{Veg_i,Z} \cdot P_{Veg_i,Z} - C_{Veg_i,Z}) \\
 s.t. \quad & M_{Fd_i}(1 - \delta) < \sum_Z S_{Fd_i,Z} \cdot Y_{Fd_i,Z} < M_{Fd_i}, \forall 1 \leq i \leq 10 \\
 & \sum_{i=1}^{10} S_{Fd_i,Z} < S_Z, \forall Z = A, B, C \\
 & M_{Veg_i}(1 - \delta) < \sum_Z S_{Veg_i,Z} \cdot Y_{Veg_i,Z} < M_{Veg_i}, \forall 1 \leq i \leq 15 \\
 & \sum_{i=1}^{15} S_{Veg_i,Z} < S_Z, \forall Z = E^1, F \\
 & Q_0 \in \mathcal{Q}
 \end{aligned}$$

6.2.2 D 地块上种植两季蔬菜时的约束规划模型

此情况下考虑 A, B, C, D^1, E^1, F 上的优化问题, D^2 和 E^2 与其他作物的种植相独立, 另建立约束规划模型。

$$\begin{aligned}
 \max \quad & X_{Q_0} + \sum_{i=1}^{10} \sum_{Z=A,B,C} S_{Fd_i,Z} (Y_{Fd_i,Z} \cdot P_{Fd_i,Z} - C_{Fd_i,Z}) \\
 & + \sum_{i=1}^{15} \sum_{Z=D^1,E^1,F} S_{Veg_i,Z} (Y_{Veg_i,Z} \cdot P_{Veg_i,Z} - C_{Veg_i,Z})
 \end{aligned}$$

$$\begin{aligned}
s.t. \quad & M_{Fd_i}(1 - \delta) < \sum_Z S_{Fd_i,Z} \cdot Y_{Fd_i,Z} < M_{Fd_i}, \forall 1 \leq i \leq 10 \\
& \sum_{i=1}^{10} S_{Fd_i,Z} < S_Z, \forall Z = A, B, C \\
& M_{Veg_i}(1 - \delta) < \sum_Z S_{Veg_i,Z} \cdot Y_{Veg_i,Z} < M_{Veg_i}, \forall 1 \leq i \leq 15 \\
& \sum_{i=1}^{15} S_{Veg_i,Z} < S_Z, \forall Z = D^1, E^1, F \\
& Q_0 \in \mathcal{Q}
\end{aligned}$$

6.3 过剩折价出售

对于过剩以 50% 原销售价格出售的情况，添加超参数 $\delta > 0$ ，在约束条件中允许作物 i 的产量超过预期销售量 ($< M_i(1 + \delta)$)，在过剩滞销的优化模型的基础上修改得到如下约束优化模型。

6.3.1 D 地块上只种植水稻时的约束规划模型

$$\begin{aligned}
\max_{Q_0 \in \mathcal{Q}} \quad & X_{Q_0} + \sum_{i=1}^{10} \sum_{Z=A,B,C} \min\{S_{Fd_i,Z} Y_{Fd_i,Z}, M_{Fd_i,Z}\} \cdot (P_{Fd_i,Z} - \frac{C_{Fd_i,Z}}{Y_{Fd_i,Z}}) \\
& + \sum_{i=1}^{10} \sum_{Z=A,B,C} \max\{(S_{Fd_i,Z} Y_{Fd_i,Z} - M_{Fd_i,Z}), 0\} \cdot (\frac{1}{2}P_{Fd_i,Z} - \frac{C_{Fd_i,Z}}{Y_{Fd_i,Z}}) \\
& + \sum_{i=1}^{15} \sum_{Z=E^1,F} \min\{S_{Veg_i,Z} Y_{Veg_i,Z}, M_{Veg_i,Z}\} \cdot (P_{Veg_i,Z} - \frac{C_{Veg_i,Z}}{Y_{Veg_i,Z}}) \\
& + \sum_{i=1}^{15} \sum_{Z=E^1,F} \max\{(S_{Veg_i,Z} Y_{Veg_i,Z} - M_{Veg_i,Z}), 0\} \cdot (\frac{1}{2}P_{Veg_i,Z} - \frac{C_{Veg_i,Z}}{Y_{Veg_i,Z}}) \\
s.t. \quad & M_{Fd_i}(1 - \delta) < \sum_Z S_{Fd_i,Z} \cdot Y_{Fd_i,Z} < M_{Fd_i}(1 + \delta), \forall 1 \leq i \leq 10 \\
& \sum_{i=1}^{10} S_{Fd_i,Z} < S_Z, \forall Z = A, B, C \\
& M_{Veg_i}(1 - \delta) < \sum_Z S_{Veg_i,Z} \cdot Y_{Veg_i,Z} < M_{Veg_i}(1 + \delta), \forall 1 \leq i \leq 15 \\
& \sum_{i=1}^{15} S_{Veg_i,Z} < S_Z, \forall Z = E^1, F
\end{aligned}$$

6.3.2 D 地块上种植两季蔬菜时的约束规划模型

$$\begin{aligned}
& \max_{Q_0 \in \mathcal{Q}} X_{Q_0} + \sum_{i=1}^{10} \sum_{Z=A,B,C} \min\{S_{Fd_i,Z} Y_{Fd_i,Z}, M_{Fd_i,Z}\} \cdot (P_{Fd_i,Z} - \frac{C_{Fd_i,Z}}{Y_{Fd_i,Z}}) \\
& + \sum_{i=1}^{10} \sum_{Z=A,B,C} \max\{(S_{Fd_i,Z} Y_{Fd_i,Z} - M_{Fd_i,Z}), 0\} \cdot (\frac{1}{2} P_{Fd_i,Z} - \frac{C_{Fd_i,Z}}{Y_{Fd_i,Z}}) \\
& + \sum_{i=1}^{15} \sum_{Z=D^1, E^1, F} \min\{S_{Veg_i,Z} Y_{Veg_i,Z}, M_{Veg_i,Z}\} \cdot (P_{Veg_i,Z} - \frac{C_{Veg_i,Z}}{Y_{Veg_i,Z}}) \\
& + \sum_{i=1}^{15} \sum_{Z=D^1, E^1, F} \max\{(S_{Veg_i,Z} Y_{Veg_i,Z} - M_{Veg_i,Z}), 0\} \cdot (\frac{1}{2} P_{Veg_i,Z} - \frac{C_{Veg_i,Z}}{Y_{Veg_i,Z}}) \\
& s.t. M_{Fd_i}(1 - \delta) < \sum_Z S_{Fd_i,Z} \cdot Y_{Fd_i,Z} < M_{Fd_i}(1 + \delta), \forall 1 \leq i \leq 10 \\
& \sum_{i=1}^{10} S_{Fd_i,Z} < S_Z, \forall Z = A, B, C \\
& M_{Veg_i}(1 - \delta) < \sum_Z S_{Veg_i,Z} \cdot Y_{Veg_i,Z} < M_{Veg_i}(1 + \delta), \forall 1 \leq i \leq 15 \\
& \sum_{i=1}^{15} S_{Veg_i,Z} < S_Z, \forall Z = D^1, E^1, F
\end{aligned}$$

6.4 D^2, E^2 上的约束优化模型

根据附件说明, E^2 中只种植 Fu , 与其他地块种植的作物完全互斥, 因此可以单独考虑 E^2 的种植。又因为 E 的面积完全均等, 主要工作即为分配不能连茬, 同时满足较大利润。

过剩滞销:

$$\begin{aligned}
& \max \sum_{i=1}^4 S_{Fu_i} (Y_{Fu_i} \cdot P_{Fu_i} - C_{Fu_i}) \\
& s.t. M_{Fu_i}(1 - \delta) < S_{Fu_i} \cdot Y_{Fu_i} < M_{Fu_i}, \forall 1 \leq i \leq 4 \\
& \sum_{i=1}^4 S_{Fu_i} < S_E
\end{aligned}$$

过剩以 0.5 价格出售:

$$\begin{aligned}
& \max \sum_{i=1}^4 \min\{S_{Fu_i} Y_{Fu_i}, M_{Fu_i}\} \cdot (P_{Fu_i} - \frac{C_{Fu_i}}{Y_{Fu_i}}) \\
& + \sum_{i=1}^4 \max\{(S_{Fu_i} Y_{Fu_i} - M_{Fu_i}), 0\} \cdot (\frac{1}{2}P_{Fu_i} - \frac{C_{Fu_i}}{Y_{Fu_i}}) \\
& s.t. M_{Fu_i}(1 - \delta) < S_{Fu_i} \cdot Y_{Fu_i} < M_{Fu_i}(1 + \delta), \forall 1 \leq i \leq 4 \\
& \sum_{i=1}^4 S_{Fu_i} < S_E
\end{aligned}$$

当采取 D 地块上种植两季蔬菜时, D^2 上只能种植 $Veg_{16,17,18}$, 与其他作物互斥, 因此有类似的约束优化模型:

过剩滞销:

$$\begin{aligned}
& \max \sum_{i=16}^{18} S_{Veg_i} (Y_{Veg_i} \cdot P_{Veg_i} - C_{Veg_i}) \\
& s.t. M_{Veg_i}(1 - \delta) < S_{Veg_i} \cdot Y_{Veg_i} < M_{Veg_i}, \forall 16 \leq i \leq 18 \\
& \sum_{i=16}^{18} S_{Veg_i} < S_D
\end{aligned}$$

过剩以 0.5 价格出售:

$$\begin{aligned}
& \max \sum_{i=16}^{18} \min\{S_{Veg_i} Y_{Veg_i}, M_{Veg_i}\} \cdot (P_{Veg_i} - \frac{C_{Veg_i}}{Y_{Veg_i}}) \\
& + \sum_{i=16}^{18} \max\{(S_{Veg_i} Y_{Veg_i} - M_{Veg_i}), 0\} \cdot (\frac{1}{2}P_{Veg_i} - \frac{C_{Veg_i}}{Y_{Veg_i}}) \\
& s.t. M_{Veg_i}(1 - \delta) < S_{Veg_i} \cdot Y_{Veg_i} < M_{Veg_i}(1 + \delta), \forall 16 \leq i \leq 18 \\
& \sum_{i=16}^{18} S_{Veg_i} < S_D
\end{aligned}$$

6.5 轮植方案

在确定每年各个地块的作物种植面积之后, 需要满足作物的种植不连茬. 采取轮植的做法。

假设地块类型 Z 下有 n_Z 个地块:

1. $Z = A, B, C$: 假设在 Z 上种植 k 种作物, 将 Z 均等分为 m 个子块, 每个子块尽量包含完整的地块, 之后进行轮换种植即可;
2. $Z = D$: 同上;
3. $Z = E$: 无需考虑连茬问题, E^1, E^2 种植作物不同;

4. $Z = F$: 类似。

具体的轮植方案可以非常简单, 具有实际可操作性:

对于 Z 地块类型, 假设其可以种植的作物为 $i_1, \dots, i_m$, 种植面积需求分别为 $S_{i_1,Z}, \dots, S_{i_m,Z}$, 首先重新排序得到升序序列: $S_{(i_1),Z}, S_{(i_2),Z}, \dots, S_{(i_m),Z}$, 再取累计分布的中点 $S_{(i_0),Z}$ 使得 $\sum_{j=1}^{i_0} S_{(i_j),Z} \approx \frac{1}{2} \sum_{j=1}^m S_{(i_m),Z}$, 对于 Z 地块类型下的所有地块 $Z_1, \dots, Z_n$, 设其对应面积为 $S_{Z_1}, \dots, S_{Z_n}$, 同样做升序排列得到: $S_{(Z_1)}, \dots, S_{(Z_n)}$, 进而取 j_0 使得: $\sum_{i=1}^{j_0} S_{(Z_i)} \approx \frac{1}{2} \sum_{i=1}^n S_{Z_i}$ 。

进而, 将种植的作物划分为两组 $\mathcal{S}_{I_1,Z}, \mathcal{S}_{I_2,Z}$, 种植的 Z 类型地块划分为两组 $\mathcal{S}_{1,Z}, \mathcal{S}_{2,Z}$, 交换种植即可。

七、问题二模型建立与求解

7.1 对各类作物变动的分析

作物	$M_{i,Z}^t$	$C_{i,Z}^t$	$Y_{i,Z}^t$	$P_{i,Z}^t$
Fd_1, Fd_2	+5% ~ 10% 平均年增长率	稳定	±10% 每年	稳定
$Fd_i (i \neq 1, 2)$	±5% 相对于 2023	稳定	±10% 每年	稳定
Veg	±5% 相对于 2023	稳定	±10% 每年	+5% 平均年增长率
$Fu_i (i \neq 4)$	±5% 相对于 2023	稳定	±10% 每年	-1% ~ -5%
Fu_4	±5% 相对于 2023	稳定	±10% 每年	-5%

表 1

分析可得如下结论:

1. A, B, C 上只能够种植 Fd 和 Fd^d , 而 Fd_1, Fd_2 的 $M_{i,Z}^t$ 上升要高于其他 Fd, Fd^d , 因此有较大的影响;
2. 对于 Veg 而言, 其整体上价格都在增长, 因此 D, E, F 的分配方案基本是不变的, 但是将会更倾向于种植亩利润大的作物;
3. 对于 Fu 而言, 其只能够种植在 E^2 上, 因此各项参数变动的影响较少。

设第 t 年作物 i 在地块 Z 上种植面积浮动占比变化为 $\delta_{i,Z}^t \in (-1, U)$, 其中 U 为一常数, 表示其种植面积不可能无限制增长。

则约束条件为当年的总净面积变化不能超出地块总面积:

$$\begin{aligned}
X_i^t - X_i^{t-1} &= \sum_Z S_{i,Z}^t (Y_{i,Z}^t \cdot P_{i,Z}^t - C_{i,Z}^{t-1}) - \sum_Z S_{i,Z}^{t-1} (Y_{i,Z}^{t-1} \cdot P_{i,Z}^t - C_{i,Z}^{t-1}) \\
&= \sum_Z (1 + \delta_{i,Z}^t) S_{i,Z}^{t-1} [(1 + \Delta y_i^t) Y_{i,Z}^{t-1} (1 + \Delta p_i^t) P_{i,Z}^{t-1} - C_{i,Z}^t] \\
&\quad - \sum_Z S_{i,Z}^{t-1} (Y_{i,Z}^{t-1} \cdot P_{i,Z}^t - C_{i,Z}^{t-1}) \\
&= \sum_Z [(1 + \delta_{i,Z}^t)(1 + \Delta y_i^t)(1 + \Delta p_i^t) - 1] S_{i,Z}^{t-1} Y_{i,Z}^{t-1} P_{i,Z}^{t-1} \\
&\quad - \sum_Z \delta_{i,Z}^t S_{i,Z}^{t-1}
\end{aligned}$$

说明：按照题目说明， $C_{i,Z}^t$ 并无明显变化, 认为其稳定, 记作 $C_{i,Z}$ 。

显然其中有两个重要的因子:

1. 利润增长因子: $\alpha_{i,Z}^t = (1 + \delta_{i,Z}^t)(1 + \Delta y_i^t)(1 + \Delta p_i^t)$;
2. 利润降低因子: $\beta_{i,Z}^t = -\delta_{i,Z}^t$ 。

以及基线: $B_{i,Z,t-1}^1 = S_{i,Z}^{t-1} Y_{i,Z}^{t-1} P_{i,Z}^{t-1}$, $B_{i,Z,t-1}^2 = S_{i,Z}^{t-1}$ 。

利润重新写作:

$$\Delta X_i^t = \sum_Z \alpha_{i,Z}^t B_{i,Z,t-1}^1 + \beta_{i,Z}^t B_{i,Z,t-1}^2$$

目标函数即为: $\sum_{i=1}^I \Delta X_i^t$, 采取合适的策略优化即可。

八、问题三模型建立与求解

8.1 作物的可替代性与互补性

根据所查找文献 [1, 2], 我们认为如下作物具有可替代关系:

1. 豆类 (粮食) 替代作物: $Fd_{i=1,\dots,5}^d$, 记为 Fd^d ; 这些豆类作物在食用和加工上具有一定的替代性。它们都可以作为食品原料、制作豆制品或用于烹饪, 消费者可以根据个人口味和需求进行选择;
2. 豆类 (蔬菜) 替代作物: $Veg_{i=1,2,3}^d$, 记为 Veg^d ;
3. 粮食 (非豆类) 替代作物: $Fd_{i=1,2}$, 分别是小麦 (6) 和玉米 (7), 两者都是重要的饲料原料, 同时在食品加工中也有广泛应用, 同时从 2023 年的销售数据中也能看出两者的需求近似. (170840.0, 132750.0);
4. 蔬菜 (萝卜) 替代作物: $Veg_{16,17,18}$ 大白菜、白萝卜、红萝卜, 记为 Veg^{rab} ;

5. 由于蔬菜作物在烹饪和食用上具有很强的替代性。消费者可以根据季节、价格和人口口味来选择不同的蔬菜品种。因此不构成太强的替代关系。

互补性作物则是指在消费或生产过程中需要相互搭配或协同作用的商品, 分析认为主要存在主食与副食的互补性, 因此假设粮食类 Fd 与 Veg 之间存在互补关系:

1. 主食: $Fd_{i=1,2}$ (小麦、玉米) 和 Fd_{11} (水稻); Fd^{main} ;
2. 副食: 豆类 Fd^d , Veg^d 和蔬菜 Veg 。

8.2 采取随机模拟决定种植策略

假设 $M_{i,Z}^t = f(P_{i,Z}^t)$, 并假设所有的地块和时间都有相同的销售规律: $M_i = f(P_i)$, $P_i = g(M_i, C_i)$, 又因为问题二中种植成本并未给出波动情况, 我们假设种植成本 C_i 稳定, 因此销售关系只需要考虑 $P - M$ 的相互影响。

根据经济学相关知识, 预期销售量 M 与价格 P 变动的关系通常涉及供需模型和弹性分析, 农产品需求价格弹性较低。销售量 M 与销售价格 P 通常呈现负相关性, 对与农产品亦然。

当产品为农产品时, 一个显著特征是价格下降导致的需求增加非常少, 也即供给增加、价格下降并不会导致收益和利润的增长, 反而会导致下降, 这是经济学中的“谷贱伤农”的原理解释。(因此若 $M = \alpha \cdot P + \beta$, α 应当较小。)

基于在问题二中建立的模型, 对于作物 i , 若其在时间 t 时预期销售量变化为 Δm_i^t , 则当年的预期销售量变化为 $\Delta M_i^t = M_i^{t-1} \Delta m_i^t$, 在 C_i 稳定的情况下, 将会造成的价格变动为: $\Delta P_i^t = f(M_i^{t-1} + \Delta M_i^t) - f(M_i^t)$ 。

地块总共可以分为如下 2 种类型: 粮食类地块 $Z_{Fd} = \{A, B, C\}$; 蔬菜类地块 $Z_{Veg} = \{D^1, D^2, E^1, F\}$ 。

对于 D, E^2 (水稻、食用菌) 因为与其他地块种植的作物不构成交集, 并且地块种植的作物固定, 因此在相应年份无需考虑。

按照可替代性, 假设 Z_{Fd} 有如下 m 类可替代作物: $FD_1, \dots, FD_m$; Z_{Veg} 有如下 n 类可替代作物 $VEG_1, \dots, VEG_n$ 。

对于每个可替代类, FD_i, VEG_j , 在替代类内以正态分布 $\mathcal{N}(0, 0.1)$ 随机扰动各个作物的种植面积变化, 并限制种植面积变化之和不变。

在预期销售量 M_i^t 不改变的基础上, 假设产量为 N_i^t , $M_i^t - N_i^t$ 将会直接影响销售价格 P_i^t 和实际销售量 $M_{(i)}^t$ 。

九、模型的评价与推广

9.1 模型优点

1. 将问题中多个因素拆解为不同的独立阶段，分别进行优化建模，所得出的结果便于管理和施行；
2. 对于豆类作物三年一种、不能连茬等限制，将问题转化张量下的 0 – 1 整数规划求解，扩展了整数规划方法。

9.2 模型缺点

1. 采用随机模拟的方法求解复杂因素下的解，该方法推广性较差，且只能够取得局部的解；
2. 多阶段的优化得到的解在阶段间存在依赖性的情况下并不是次最优解。

参考文献

- [1] 宋长鸣, 李崇光. 农产品市场的整合与替代研究——以蔬菜为例 [J]. 中国农村经济, 2012, (11): 78-87+96.
- [2] 郭燕枝, 王小虎, 孙君茂. 华北平原地下水漏斗区马铃薯替代小麦种植及由此节省的水资源量估算 [J]. 中国农业科技导报, 2014, 16(06): 159-163. DOI: 10.13304/j.nykjdb.2014.378.
- [3] 王东明, 夏壁灿, 李子明. 计算机代数 [M]. 2 版. 北京: 清华大学出版社, 2007: 133-150.
- [4] 张丹丽, 曹锦, 高彦杰. 多目标进化算法研究综述 [J]. 科技创新与应用, 2021, 11(28): 69-71+74.

附录 A 支撑材料

所属部分	文件名	简介
Data	CropSize.txt	2023 年各个作物在各个地块的种植面积
	data_common_fd.txt	常规粮食类作物的销售数据
	data_common_veg.txt	常规蔬菜类作物的销售数据
	data_dou_abc.txt	粮食类(豆类)作物的销售数据
	data_dou_def.txt	蔬菜类(豆类)作物的销售数据
	data_fu.txt	食用菌作物的销售数据
	data_rab_veg.txt	蔬菜类(胡萝卜)的销售数据
	data_rice_fd.txt	粮食类(水稻)的销售数据
	dfd_vecs.txt	2023 年豆类作物按地块种植情况
	Fields_Fd.txt	粮食类地块面积分布
	Fields_Size_Nums.txt	各类型地块面积与地块个数
	CropCommonFdVeg.py	对于常规作物的分析
	CropDou.py	对于豆类作物的分析
	CropOthers.py	对于其他作物的分析
	CropSize.py	对于作物需求的分析
Model	FieldsDiv.py	对于地块面积的分析
	FutureCrop.py	对于作物未来销售变化的分析
	DouScheme_2.py	采用 Pulp 求解豆类作物种植面积分配
	DouScheme.py	采用 0-1 整数规划求解豆类作物面积分配
	FuScheme.py	求解食用菌种植面积分配
	MonteCarlo.py	采取随机模拟进行种植面积分配
	NSGA-II.py	采取 NSGA-II 求解种植面积
Result	RabScheme_GA.py	采取遗传算法求解种植面积
	RabScheme.py	求解萝卜类作物种植面积
	result1_1.xlsx	种植方案
	result1_2.xlsx	种植方案
	result2.xlsx	种植方案

支撑材料主要分为两个部分：Model 部分建立模型，主要用于求解，Data 部分进行数据汇总与提取，所有较长的中间结果以.txt 或者.csv 格式存储在 Data 中。

附录 B 各类型地块面积与分布

地块类型	总地块面积	地块个数
A	365.0	6
B	619.0	14
C	108.0	6
D	109.0	8
E	9.6	16
F	2.4	4

附录 C 各个作物在 2023 年的产量、定价与种植面积

3.1 粮食类作物 (豆类)

作物编号	作物名称	$Y_{i,D1}$	$C_{i,D1}$	$P_{i,D1}$	$Y_{i,E1}$	$C_{i,E1}$	$P_{i,E1}$
17	豇豆	3000	2000	8.0	3600	2400	8.0
18	刀豆	2000	1000	6.75	2400	1200	6.75
19	芸豆	3000	2000	6.5	3600	2400	6.5

作物编号	$Y_{i,F2}$	$C_{i,F2}$	$P_{i,F2}$	M	$S_{\max}$	$S_{\min}$	S
17	3200	2640	9.6	36480.0	12.2	10.1	11.8
18	2200	1320	8.1	26880.0	13.4	11.2	13.2
19	3200	2640	7.8	6480.0	2.2	1.8	1.8

3.2 粮食类作物 (非豆类)

作物编号	作物名称	$Y_{i,A}$	$Y_{i,B}$	$Y_{i,Z}$	C	P	M	$S_{\max}$	$S_{\min}$	S
6	小麦	800	760	720	450	3.5	170840.0	237.3	213.6	222.0
7	玉米	1000	950	900	500	3.0	132750.0	147.5	132.8	135.0
8	谷子	400	380	360	360	6.75	71400.0	198.3	178.5	185.0
9	高粱	630	600	570	400	6.0	30000.0	52.6	47.6	50.0
10	黍子	525	500	475	360	7.5	12500.0	26.3	23.8	25.0
11	荞麦	110	105	100	350	40.0	1500.0	15.0	13.6	15.0
12	南瓜	3000	2850	2700	1000	1.5	35100.0	13.0	11.7	13.0
13	红薯	2200	2100	2000	2000	3.25	36000.0	18.0	16.4	18.0
14	莠麦	420	400	380	400	5.5	14000.0	36.8	33.3	35.0
15	大麦	525	500	475	350	3.5	10000.0	21.1	19.0	20.0

3.3 蔬菜类作物 (豆类)

作物编号	作物名称	Y(D1)	C(D1)	P(D1)	Y(E1)	C(E1)	P(E1)
17	豇豆	3000	2000	8.0	3600	2400	8.0
18	刀豆	2000	1000	6.75	2400	1200	6.75
19	芸豆	3000	2000	6.5	3600	2400	6.5

作物编号	Y(F2)	C(F2)	P(F2)	Y	$S_{\max}$	$S_{\min}$	S
17	3200	2640	9.6	36480.0	12.2	10.1	11.8
18	2200	1320	8.1	26880.0	13.4	11.2	13.2
19	3200	2640	7.8	6480.0	2.2	1.8	1.8

3.4 蔬菜类作物 (非豆类)

作物编号	作物名称	Y(D1)	C(D1)	P(D1)	Y(E1)	C(E1)	P(E1)
20	土豆	2000	2000	3.75	2400	2400	3.75
21	西红柿	2400	2000	6.25	3000	2400	6.25
22	茄子	6400	2000	5.5	8000	2400	5.5
23	菠菜	2700	2300	5.75	3300	2700	5.75
24	青椒	2400	1600	5.25	3000	2000	5.25
25	菜花	3300	2400	5.5	4000	3000	5.5
26	包菜	3700	2900	6.5	4500	3500	6.5
27	油麦菜	4100	1600	5.0	5000	2000	5.0
28	小青菜	3200	1600	5.75	4000	2000	5.75
29	黄瓜	12000	2900	7.0	15000	3500	7.0
30	生菜	4100	1600	5.25	5000	2000	5.25
31	辣椒	1600	1000	7.25	2000	1200	7.25
32	空心菜	10000	4100	4.5	12000	5000	4.5
33	黄心菜	5000	2000	4.5	6000	2500	4.5
34	芹菜	5500	900	4.0	6600	1100	4.0

作物编号	Y(F2)	C(F2)	P(F2)	Y	$S_{\max}$	$S_{\min}$	S
20	2200	2640	4.5	30000.0	15.0	12.5	15.0
21	2700	2640	7.5	36210.0	15.1	12.1	14.9
22	7200	2640	6.6	45360.0	7.1	5.7	6.9
23	3000	3000	6.9	900.0	0.3	0.3	0.3
24	2700	2200	6.8	2610.0	1.1	0.9	0.9
25	3600	3300	6.6	3600.0	1.1	0.9	0.9
26	4100	3850	7.8	4050.0	1.1	0.9	0.9
27	4500	2200	6.0	4500.0	1.1	0.9	0.9
28	3600	2200	6.9	35480.0	11.1	8.9	10.9
29	13500	3850	8.4	13050.0	1.1	0.9	0.9
30	4500	2200	6.3	2850.0	0.7	0.6	0.6
31	1800	1300	8.7	1200.0	0.8	0.6	0.6
32	11000	5500	5.4	3600.0	0.4	0.3	0.3
33	5400	2750	5.4	1800.0	0.4	0.3	0.3
34	6000	1200	4.8	1800.0	0.3	0.3	0.3

3.5 食用菌作物

作物编号	作物名称	Y(D1)	C(D1)	P(D1)	Y(E1)	C(E1)	P(E1)
17	豇豆	3000	2000	8.0	3600	2400	8.0
18	刀豆	2000	1000	6.75	2400	1200	6.75
19	芸豆	3000	2000	6.5	3600	2400	6.5

作物编号	Y(F2)	C(F2)	P(F2)	Y	$S_{\max}$	$S_{\min}$	S
17	3200	2640	9.6	36480.0	12.2	10.1	11.8
18	2200	1320	8.1	26880.0	13.4	11.2	13.2
19	3200	2640	7.8	6480.0	2.2	1.8	1.8

附录 D 主要源代码

4.1 对于常规蔬菜与粮食作物的销售数据分析

```
import pandas as pd
import numpy as np

def to_mean(p_range):
    try:
        # 分割范围字符串并计算均值
        lower, upper = map(float, p_range.split('-'))
        return (lower + upper) / 2
    except:
        return None

def get_land_plot_type(row):
    if row['地块基础类型'] == 'F':
        return f"{row['地块基础类型']}{1 if row['种植季次'] == '第一季' else 2}"
    else:
        return row['地块基础类型']

file_path = 'file2.xlsx'
df = pd.read_excel(file_path, sheet_name=0)
df['种植地块'] = df['种植地块'].ffill()
df['地块基础类型'] = df['种植地块'].str[0]
df['地块类型'] = df.apply(get_land_plot_type, axis=1)

df_1 = pd.read_csv('Fields_Size_Vec.csv')
mapping = dict(zip(df_1['地块名称'], df_1['地块面积/亩']))
df['地块面积/亩'] = df['种植地块'].map(mapping)
df[['种植地块', '地块类型', '地块基础类型', '地块面积/亩']].to_csv('Fields_Size_Vec_Extend.csv',
    index=False)

area_by_crop = df.groupby(['作物编号', '地块类型'])['种植面积/亩'].sum().unstack(fill_value=0)
area_by_crop['总种植面积'] = area_by_crop.sum(axis=1)
area_by_crop.round(1).to_csv('CropSizeExtend.csv', encoding='utf-8')

fv_index = list(range(6, 16)) + list(range(20, 35))
fd_index = list(range(6, 16))
veg_index = list(range(20, 35))

df_size = pd.read_csv('CropSizeExtend.csv')
data_size = df_size[df_size['作物编号'].isin(fv_index)]
df_sale = pd.read_excel('file2.xlsx', sheet_name=1, nrows=108)
data_sale = df_sale[df_sale['作物编号'].isin(fv_index)]
```

```

rows_abc = []
rows_def = []

# 对于常规 Fd 进行处理
for crop_id in fd_index:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(A)': crop_data[crop_data['地块类型'] == '平旱地']['亩产量/斤'].values[0],
        'Y(B)': crop_data[crop_data['地块类型'] == '梯田']['亩产量/斤'].values[0],
        'Y(C)': crop_data[crop_data['地块类型'] == '山坡地']['亩产量/斤'].values[0],
        'C': crop_data['种植成本/(元/亩)'].iloc[0],
        'P': crop_data['销售单价/(元/斤)'].iloc[0]
    }
    rows_abc.append(row)

# 对于常规 Veg 进行处理
for crop_id in veg_index:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(D1)': crop_data[crop_data['地块类型'] == '水浇地']['亩产量/斤'].values[0],
        'C(D1)': crop_data[crop_data['地块类型'] == '水浇地']['种植成本/(元/亩)'].values[0],
        'P(D1)': crop_data[crop_data['地块类型'] == '水浇地']['销售单价/(元/斤)'].values[0],
        'Y(E1)': crop_data[crop_data['地块类型'].str.strip() ==
            '普通大棚']['亩产量/斤'].values[0],
        'C(E1)': crop_data[crop_data['地块类型'].str.strip() ==
            '普通大棚']['种植成本/(元/亩)'].values[0],
        'P(E1)': crop_data[crop_data['地块类型'].str.strip() ==
            '普通大棚']['销售单价/(元/斤)'].values[0],
        'Y(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['亩产量/斤'].values[0],
        'C(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['种植成本/(元/亩)'].values[0],
        'P(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['销售单价/(元/斤)'].values[0]
    }
    rows_def.append(row)

df_fd = pd.DataFrame(rows_abc)
df_veg = pd.DataFrame(rows_def)

# 计算  $Y = A * Y(A) + B * Y(B) + C * Y(C)$ 
merged_df = pd.merge(df_fd, data_size[['作物编号', 'A', 'B', 'C']], on='作物编号')
merged_df['Y'] = merged_df['A'] * merged_df['Y(A)'] + merged_df['B'] * merged_df['Y(B)'] +
    merged_df['C'] * merged_df['Y(C)']
# merged_df['Y'] = merged_df['A'] * merged_df['Y(A)'] + merged_df['B'] * merged_df['Y(B)'] +
    merged_df['C'] * merged_df['Y(C)']

```

```

df_fd['Y'] = merged_df['Y']

# 计算  $Y = D * Y(D1) + E * Y(E1) + F1 * Y(E1) + F2 * Y(F2)$ 
# 注意  $Y(E1) = Y(F1)$ 
merged_df = pd.merge(df_veg, data_size[['作物编号', 'D', 'E', 'F1', 'F2']], on='作物编号')
merged_df['Y'] = merged_df['D'] * merged_df['Y(D1)'] + merged_df['E'] * merged_df['Y(E1)'] +
    merged_df['F1'] * merged_df['Y(E1)'] + merged_df['F2'] * merged_df['Y(F2)']
df_veg['Y'] = merged_df['Y']

data_size = data_size.reset_index(drop=True)
# 计算种植 作物的最大 / 最小种植亩数需求
# 这里仍然可以发现 Y(C) 最小 Y(A) 最大; Y(D1) 最小 Y(E1) 最大
df_fd['S_max'] = (df_fd['Y'] / df_fd['Y(C)']).round(1)
df_fd['S_min'] = (df_fd['Y'] / df_fd['Y(A)']).round(1)
df_fd['S'] = data_size['总种植面积'].iloc[0:10]
df_veg['S_max'] = (df_veg['Y'] / df_veg['Y(D1)']).round(1)
df_veg['S_min'] = (df_veg['Y'] / df_veg['Y(E1)']).round(1)
df_veg['S'] = data_size['总种植面积'].iloc[10:].reset_index(drop=True)

# 销售区间转化为平均值
df_fd['P'] = df_fd['P'].apply(to_mean)
df_veg['P(D1)'] = df_veg['P(D1)'].apply(to_mean)
df_veg['P(E1)'] = df_veg['P(E1)'].apply(to_mean)
df_veg['P(F2)'] = df_veg['P(F2)'].apply(to_mean)

# 结果
df_fd.to_csv('data_common_fd.txt', sep=' ', encoding='utf-8', index=False)
df_veg.to_csv('data_common_veg.txt', sep=' ', encoding='utf-8', index=False)

```

4.2 对于豆类作物销售数据的分析

```

import pandas as pd
import numpy as np

def bin_a(indices, length=26):
    binary_array = np.zeros(length, dtype=int)
    # 将指定索引位置设置为 1
    binary_array[indices] = 1
    return binary_array

def to_mean(p_range):
    try:
        # 分割范围字符串并计算均值
        lower, upper = map(float, p_range.split('-'))
        return (lower + upper) / 2
    
```

```

except:
    return None

dou_index = [1, 2, 3, 4, 5, 17, 18, 19]
fd_dou_id = [1, 2, 3, 4, 5]

df_size = pd.read_csv('CropSize.csv')
data_size = df_size[df_size['作物编号'].isin(dou_index)]
df_sale = pd.read_excel('file2.xlsx', sheet_name=1, nrows=108)
data_sale = df_sale[df_sale['作物编号'].isin(dou_index)]

df_d_0 = pd.read_excel('file2.xlsx', sheet_name=0, nrows=88)
df_d_0['种植地块'] = df_d_0['种植地块'].ffill()
df_d = df_d_0[df_d_0['作物类型'].isin(['粮食（豆类）', '蔬菜（豆类）'])]
df_d['Class'] = df_d['种植地块'].str[0]
df_dfd = df_d[df_d['Class'].isin(['A', 'B', 'C'])]
df_dveg = df_d[df_d['Class'].isin(['D', 'E', 'F'])]

dfd_vecs = []
for i in range(5):
    dfd_vec = bin_a(df_dfd[df_dfd['作物编号'] == i + 1].index)
    dfd_vecs.append(dfd_vec)
dfd_vecs = np.array(dfd_vecs)
np.savetxt('dfd_vecs.txt', dfd_vecs, fmt='%d')

rows_abc = []
rows_def = []

# 对作物编号 1, 2, 3, 4, 5 进行处理
for crop_id in [1, 2, 3, 4, 5]:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(A)': crop_data[crop_data['地块类型'] == '平旱地']['亩产量/斤'].values[0],
        'Y(B)': crop_data[crop_data['地块类型'] == '梯田']['亩产量/斤'].values[0],
        'Y(C)': crop_data[crop_data['地块类型'] == '山坡地']['亩产量/斤'].values[0],
        'C': crop_data['种植成本/(元/亩)'].iloc[0],
        'P': crop_data['销售单价/(元/斤)'].iloc[0]
    }
    rows_abc.append(row)

# 对作物编号 17, 18, 19 进行处理
for crop_id in [17, 18, 19]:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,

```

```

        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(D1)': crop_data[crop_data['地块类型'] == '水浇地']['亩产量/斤'].values[0],
        'C(D1)': crop_data[crop_data['地块类型'] == '水浇地']['种植成本/(元/亩)'].values[0],
        'P(D1)': crop_data[crop_data['地块类型'] == '水浇地']['销售单价/(元/斤)'].values[0],
        'Y(E1)': crop_data[crop_data['地块类型'] == '普通大棚'].str.strip() ==
            '普通大棚']['亩产量/斤'].values[0],
        'C(E1)': crop_data[crop_data['地块类型'] == '普通大棚'].str.strip() ==
            '普通大棚']['种植成本/(元/亩)'].values[0],
        'P(E1)': crop_data[crop_data['地块类型'] == '普通大棚'].str.strip() ==
            '普通大棚']['销售单价/(元/斤)'].values[0],
        'Y(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['亩产量/斤'].values[0],
        'C(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['种植成本/(元/亩)'].values[0],
        'P(F2)': crop_data[crop_data['地块类型'] == '智慧大棚']['销售单价/(元/斤)'].values[0]
    }
    rows_def.append(row)

df_abc = pd.DataFrame(rows_abc)
df_def = pd.DataFrame(rows_def)

# 计算  $Y = A * Y(A) + B * Y(B) + C * Y(C)$ 
merged_df = pd.merge(df_abc, data_size[['作物编号', 'A', 'B', 'C']], on='作物编号')
merged_df['Y'] = merged_df['A'] * merged_df['Y(A)'] + merged_df['B'] * merged_df['Y(B)'] +
    merged_df['C_y'] * merged_df['Y(C)']
# merged_df['Y'] = merged_df['A'] * merged_df['Y(A)'] + merged_df['B'] * merged_df['Y(B)'] +
#     merged_df['C'] * merged_df['Y(C)']
df_abc['Y'] = merged_df['Y']
df_abc['P'] = df_abc['P'].apply(to_mean)

# 计算  $Y = D * Y(D1) + E * Y(E1) + F * Y(E1)$ 
# 注意数据中：2023 年所有种植在智慧大棚中的豆类作物都是智慧大棚第一季
merged_df = pd.merge(df_def, data_size[['作物编号', 'D', 'E', 'F']], on='作物编号')
merged_df['Y'] = merged_df['D'] * merged_df['Y(D1)'] + merged_df['E'] * merged_df['Y(E1)'] +
    merged_df['F'] * merged_df['Y(E1)']
df_def['Y'] = merged_df['Y']
df_def['P(D1)'] = df_def['P(D1)'].apply(to_mean)
df_def['P(E1)'] = df_def['P(E1)'].apply(to_mean)
df_def['P(F2)'] = df_def['P(F2)'].apply(to_mean)

# 计算种植 5 种豆类作物的最大 / 最小种植亩数需求
df_abc['S_max'] = (df_abc['Y'] / df_abc['Y(C)']).round(1)
df_abc['S_min'] = (df_abc['Y'] / df_abc['Y(A)']).round(1)
df_abc['S'] = data_size['总种植面积'].iloc[0:5]
# 计算种植 3 类豆类作物的最大 / 最小种植亩数需求
df_def['S_max'] = (df_def['Y'] / df_def['Y(D1)']).round(1)
df_def['S_min'] = (df_def['Y'] / df_def['Y(E1)']).round(1)
df_def['S'] = data_size['总种植面积'].iloc[5:].reset_index(drop=True)

```

```
# 结果
```

```
df_abc.to_csv('data_dou_abc.txt', sep=' ', encoding='utf-8', index=False)
df_def.to_csv('data_dou_def.txt', sep=' ', encoding='utf-8', index=False)
```

4.3 对于其他作物的销售数据分析

```
import pandas as pd
import numpy as np

def to_mean(p_range):
    try:
        lower, upper = map(float, p_range.split('-'))
        return (lower + upper) / 2
    except:
        return None

rice_index = [16]
rab_index = list(range(35, 38))
fu_index = list(range(38, 42))
rrf_index = rice_index + rab_index + fu_index

df_size = pd.read_csv('CropSizeExtend.csv')
data_size = df_size[df_size['作物编号'].isin(rrf_index)]
df_sale = pd.read_excel('file2.xlsx', sheet_name=1, nrows=108)
data_sale = df_sale[df_sale['作物编号'].isin(rrf_index)]

rows_rice = []
rows_rab = []
rows_fu = []

# 对于 Fd_11 进行处理
for crop_id in rice_index:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(D)': crop_data[crop_data['地块类型'] == '水浇地']['亩产量/斤'].values[0],
        'C': crop_data['种植成本/(元/亩)'].iloc[0],
        'P': crop_data['销售单价/(元/斤)'].iloc[0]
    }
    rows_rice.append(row)

df_rice = pd.DataFrame(rows_rice)

# 计算  $Y = D2 * Y(D2)$ 
```

```

merged_df = pd.merge(df_rice, data_size[['作物编号', 'D']], on='作物编号')
merged_df['Y'] = merged_df['D'] * merged_df['Y(D)']
df_rice['Y'] = merged_df['Y']
df_rice['S_max'] = (df_rice['Y'] / df_rice['Y(D)']).round(1)
df_rice['S_min'] = (df_rice['Y'] / df_rice['Y(D)']).round(1)
df_rice['S'] = data_size['总种植面积'].iloc[0]

# 对于 Veg_16,17,18 进行处理
for crop_id in rab_index:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(D2)': crop_data[crop_data['地块类型'] == '水浇地']['亩产量/斤'].values[0],
        'C(D2)': crop_data[crop_data['地块类型'] == '水浇地']['种植成本/(元/亩)'].values[0],
        'P(D2)': crop_data[crop_data['地块类型'] == '水浇地']['销售单价/(元/斤)'].values[0],
    }
    rows_rab.append(row)

df_rab = pd.DataFrame(rows_rab)

#计算 Y = D2 * Y(D2)
merged_df = pd.merge(df_rab, data_size[['作物编号', 'D']], on='作物编号')
merged_df['Y'] = merged_df['D'] * merged_df['Y(D2)']
df_rab['Y'] = merged_df['Y']
df_rab['S_max'] = (df_rab['Y'] / df_rab['Y(D2)']).round(1)
df_rab['S_min'] = (df_rab['Y'] / df_rab['Y(D2)']).round(1)
df_rab['S'] = data_size['总种植面积'].iloc[1:4].reset_index(drop=True)

# 对于 Fu_1,2,3,4 进行处理
for crop_id in fu_index:
    crop_data = data_sale[data_sale['作物编号'] == crop_id]
    row = {
        '作物编号': crop_id,
        '作物名称': crop_data['作物名称'].iloc[0],
        'Y(E2)': crop_data[crop_data['地块类型'].str.strip() == '普通大棚']['亩产量/斤'].values[0],
        'C(E2)': crop_data[crop_data['地块类型'].str.strip() == '普通大棚']['种植成本/(元/亩)'].values[0],
        'P(E2)': crop_data[crop_data['地块类型'].str.strip() == '普通大棚']['销售单价/(元/斤)'].values[0],
    }
    rows_fu.append(row)

df_fu = pd.DataFrame(rows_fu)

#计算 E = E * Y(E2)

```

```

merged_df = pd.merge(df_fu, data_size[['作物编号', 'E']], on='作物编号')
merged_df['Y'] = merged_df['E'] * merged_df['Y(E2)']
df_fu['Y'] = merged_df['Y']
df_fu['S_max'] = (df_fu['Y'] / df_fu['Y(E2)']).round(1)
df_fu['S_min'] = (df_fu['Y'] / df_fu['Y(E2)']).round(1)
df_fu['S'] = data_size['总种植面积'].iloc[4:].reset_index(drop=True)

df_rice['P'] = df_rice['P'].apply(to_mean)
df_rab['P(D2)'] = df_rab['P(D2)'].apply(to_mean)
df_fu['P(E2)'] = df_fu['P(E2)'].apply(to_mean)

# 结果
df_rice.to_csv('data_rice_fd.txt', sep=' ', encoding='utf-8', index=False)
df_rab.to_csv('data_rab_veg.txt', sep=' ', encoding='utf-8', index=False)
df_fu.to_csv('data_fu.txt', sep=' ', encoding='utf-8', index=False)

```

4.4 对于粮食种植面积的分析

```

import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

# 附件 2
file_path = 'file2.xlsx'
df = pd.read_excel(file_path, sheet_name=0)
df['种植地块'] = df['种植地块'].ffill()

# 提取地块类型 (A,B,C,D,E,F)
df['地块类型'] = df['种植地块'].str[0]
df[['种植地块', '地块类型']].to_csv('FieldIndex.csv', index=False)

# 按作物编号和地块类型分类并计算种植面积
area_by_crop = df.groupby(['作物编号', '地块类型'])['种植面积/亩'].sum().unstack(fill_value=0)

plt.rcParams['font.family'] = 'SimHei'

# 热力图
sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))
sns.heatmap(area_by_crop, annot=True, cmap="YlGnBu", cbar=True)
# plt.title('不同地块类型上各作物编号的种植面积分布')
# plt.xlabel('地块类型')
# plt.ylabel('作物编号')
plt.title('Block\'s Size')

```

```

plt.xlabel('Block Types')
plt.ylabel('Crop Types')
plt.tight_layout()
plt.show()

area_by_crop['总种植面积'] = area_by_crop.sum(axis=1)
output_file = 'CropSize.txt'
area_by_crop.round(1).to_csv(output_file, sep='\t', encoding='utf-8')
area_by_crop.round(1).to_csv('CropSize.csv', encoding='utf-8')

plt.rcParams['font.family'] = 'SimHei'

# 层叠图
plt.figure(figsize=(10, 6))
area_by_crop.drop(columns=['总种植面积']).plot(kind='bar', stacked=True, colormap='Paired',
        figsize=(12, 7))
plt.title('各作物在不同地块上的种植面积层叠图')
plt.xlabel('作物编号')
plt.ylabel('种植面积 (亩)')
plt.xticks(rotation=45, ha='right')
plt.legend(title="地块类型", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.savefig("CropSize.png", bbox_inches="tight")
plt.show()

```

4.5 豆类作物种植策略

```

import pulp
import numpy as np
import pandas as pd

def solve_problem(I, T, S, beta, S_min, S_max, Q_initial):
    # 创建 pulp 的 LpProblem 实例
    prob = pulp.LpProblem("BinaryTensorOptimization", pulp.LpMinimize)
    # 创建决策变量 q_its (I x T x S)
    q = pulp.LpVariable.dicts("q", ((i, t, s) for i in range(I) for t in range(T) for s in
        range(S)),
        cat="Binary")
    # 设置目标函数为 0 (没有明确的优化目标)
    prob += 0
    # 添加约束条件:
    # 1. 对于 t=1,...,6 保证 sum(q_{its} for t=t,...,t+2) >= 1
    for i in range(I):
        for s in range(S):
            for t in range(6): # t=1 到 t=6

```

```

        prob += pulp.lpSum([q[i, t_, s] for t_ in range(t, min(t+3, T))]) >= 1
# 2. 对于 t=1,...,7 保证 sum(q_{its} for t=t,...,t+1) <= 1
for i in range(I):
    for s in range(S):
        for t in range(7): # t=1 到 t=7
            prob += pulp.lpSum([q[i, t_, s] for t_ in range(t, min(t+2, T))]) <= 1
# 3. 对于 i=1,...,I 和 t=2,...,8 保证 S_min <= sum(q_{its} * beta_s for s) <= S_max
for i in range(I):
    for t in range(1, 8): # t=2 到 t=8
        prob += pulp.lpSum([q[i, t, s] * beta[s] for s in range(S)]) >= S_min[i]
        prob += pulp.lpSum([q[i, t, s] * beta[s] for s in range(S)]) <= S_max[i]

# 确定 q_{i1s} 的初始值
for i in range(I):
    for s in range(S):
        prob += q[i, 0, s] == Q_initial[i, 0, s]
# 为 t=8 添加约束条件 (以确保该步的 q 值存在)
for i in range(I):
    for s in range(S):
        prob += pulp.lpSum([q[i, T-1, s]]) >= 0 # 添加非负约束
# 求解问题
prob.solve()

# 打印求解器状态
print(f"Solver Status: {pulp.LpStatus[prob.status]}")

if pulp.LpStatus[prob.status] != 'Optimal':
    print("No optimal solution found.")
    return None

# 提取解并存储在列表中
solution = np.zeros((I, T, S), dtype=int)
for i in range(I):
    for t in range(T):
        for s in range(S):
            # 检查 q[i, t, s] 是否有定义值
            q_value = pulp.value(q[i, t, s])
            if q_value is None:
                print(f"Variable q[{i},{t},{s}] is None, check your constraints.")
                return None
            solution[i, t, s] = int(q_value)

return solution

def distribute_values(dou_size, T_row):
    non_zero_indices = np.where(T_row > 0)[0]
    total = dou_size

```

```

new_row = np.zeros_like(T_row)

# 找到每个非零位置的最大可填充值
max_fill = T_row[non_zero_indices]

# 初始化填充值, 保证每个位置非零, 先分配一个很小的初始值
initial_fill = np.full(len(non_zero_indices), 0.01)

# 剩余的值需要分配
remaining_total = total - np.sum(initial_fill)

# 分配剩余的值, 但保证不超过T_row的相应位置并且总和是dou_size
fill_values = np.random.rand(len(non_zero_indices))
fill_values = fill_values / fill_values.sum() * remaining_total

# 确保填充的值不超过对应位置的最大值
fill_values = np.minimum(fill_values + initial_fill, max_fill)

# 如果仍然没达到total, 按比例调整填充
while np.sum(fill_values) < total:
    remaining = total - np.sum(fill_values)
    adjustment = np.random.rand(len(non_zero_indices))
    adjustment = adjustment / adjustment.sum() * remaining
    fill_values = np.minimum(fill_values + adjustment, max_fill)

# 将计算出的值填充到new_row中
new_row[non_zero_indices] = fill_values
return new_row

def create_new_T(dou_size, T_dou):
    new_T = np.zeros_like(T_dou)

    for i in range(len(dou_size)):
        new_T[i] = distribute_values(dou_size[i], T_dou[i])

    return new_T

df = pd.read_csv('../Data/Fields_Size_Vec.csv')
beta = df["地块面积/亩"][:26].values
df_S = pd.read_csv('../Data/data_dou_abc.txt', sep='\s+')
df_dfd = df_S[['S_max', 'S_min', 'S']]
df_dfd['S_'] = (df_dfd['S_max'] + df_dfd['S_min']) / 2
df_Fd = pd.read_csv('../Data/Fields_Fd.txt', sep='\s+')
S_max = df_S["S_max"].values
S_min = df_S["S_min"].values

# 调整 S_max 和 S_min

```

```

delta1 = 0.1
delta2 = 20
S_max = S_max * (1 + delta2)
S_min = S_min * (1 - delta1)

I = 5 # 样本数量
T = 8 # 时间步数
S = 26 # 维度

Q_initial = np.loadtxt('../Data/dfd_vecs.txt', dtype=int).reshape(5,1,-1)

# 求解问题
solution = solve_problem(I, T, S, beta, S_min, S_max, Q_initial)
print(solution)

T_dou = []
T_dou_size = []
for i in range(T):
    T_dou.append(solution[:,i,:] * beta)
for i in range(T):
    for j in range(I):
        T_dou_size.append(T_dou[i][j].sum())

dou_size = df_dfd['S_'].values

DouScheme = []
for i in range(1, 8):
    DouScheme.append(create_new_T(dou_size, T_dou[i]))
    print(f'{i} finished')

DouScheme = np.array(DouScheme)

```

4.6 对于地块面积分布的分析与可视化

```

import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

matplotlib.rcParams['font.family'] = 'SimHei'

# 附件 1
file_path = 'file1.xlsx'
df = pd.read_excel(file_path, sheet_name=0, usecols="A:C")

```

```

df2 = pd.read_excel(file_path, sheet_name=1)

# 分类地块为 A, B, C, D, E, F
df['Class'] = df['地块名称'].str[0]
df_Fd = df[df['Class'].isin(['A', 'B', 'C'])].sort_values(by='地块面积/亩', ascending=False)
df_Fd_A = df_Fd[df_Fd['Class'] == 'A']
df_Fd_B = df_Fd[df_Fd['Class'] == 'B']
df_Fd_C = df_Fd[df_Fd['Class'] == 'C']
df_Fd.to_csv('Fields_Fd.txt', sep=' ', encoding='utf-8', index=False)

# 不同地块类型的总面积、块数
total_area_df = df.groupby('Class')['地块面积/亩'].sum().reset_index()
total_area_df.rename(columns={'地块面积/亩': '总地块面积'}, inplace=True)
count_df = df.groupby('Class').size().reset_index(name='地块个数')
grouped_df = pd.merge(total_area_df, count_df, on='Class')
grouped_df.to_csv('Fields_Size_Nums.txt', sep=' ', encoding='utf-8', index=False)

# 不同地块面积占比
plt.figure(figsize=(10, 6))
sns.boxplot(x='Class', y='地块面积/亩', data=df)
plt.title('不同地块类别的面积分布')
plt.xlabel('地块类别')
plt.ylabel('面积 (亩)')
plt.savefig("BlocksSize.png", bbox_inches="tight")
plt.show()

# 创建区间边界
bin_edges = np.linspace(df_Fd['地块面积/亩'].min(), df_Fd['地块面积/亩'].max(), num=5)

plt.rcParams['font.family'] = 'SimHei'

# 直方图
plt.figure(figsize=(10, 6))
sns.histplot(df_Fd['地块面积/亩'], kde=False, bins=bin_edges,
             color=sns.color_palette('Paired')[0])
plt.title('地块面积/亩 直方图')
plt.xlabel('地块面积/亩')
plt.ylabel('地块数量')
plt.savefig('FieldsFdSize.png', bbox_inches="tight")
plt.show()

```

4.7 食用菌种植策略

```

from scipy.optimize import minimize
import numpy as np

```

```

import pandas as pd

def surplus_unsold(Y_fu, P_fu, C_fu, M_fu, S_E, delta):
    def objective(S_fu):
        return -np.sum(S_fu * (Y_fu * P_fu - C_fu)) # 最大化收益

    # 约束
    constraints = [
        {'type': 'ineq', 'fun': lambda S_fu: S_E - np.sum(S_fu)}, #  $S_{fu} < S_E$ 
        *[
            {'type': 'ineq', 'fun': lambda S_fu, i=i: S_fu[i] * Y_fu[i] - M_fu[i] * (1 - delta)}
            for i in range(4)
        ], #  $M_{fu,i}(1-\delta) < S_{fu,i} * Y_{fu,i}$ 
        *[
            {'type': 'ineq', 'fun': lambda S_fu, i=i: M_fu[i] - S_fu[i] * Y_fu[i]} for i in
            range(4)
        ] #  $S_{fu,i} * Y_{fu,i} < M_{fu,i}$ 
    ]

    # 初始值: 根据可行区间设置初始值
    S_fu_initial = np.array([1.6, 1.6, 1.6, 3.8]) # 让初始值更接近可行范围

    result = minimize(objective, S_fu_initial, constraints=constraints, method='SLSQP',
                      bounds=[(0, None)] * 4)

    print(result) # 打印优化结果和状态, 便于调试

    if result.success:
        return result.x
    else:
        raise ValueError("Optimization did not converge")

def surplus_discount(Y_fu, P_fu, C_fu, M_fu, S_E, delta):
    # 定义目标函数
    def objective(S_fu):
        profit_normal = np.minimum(S_fu * Y_fu, M_fu) * (P_fu - C_fu / Y_fu)
        profit_discount = np.maximum(S_fu * Y_fu - M_fu, 0) * (0.5 * P_fu - C_fu / Y_fu)
        return -(np.sum(profit_normal) + np.sum(profit_discount)) # 最大化收益

    # 约束条件
    constraints = [
        {'type': 'ineq', 'fun': lambda S_fu: S_E - np.sum(S_fu)}, #  $S_{fu} < S_E$ 
        *[
            {'type': 'ineq', 'fun': lambda S_fu, i=i: S_fu[i] * Y_fu[i] - M_fu[i] * (1 - delta)}
            for i in range(4)
        ], #  $M_{fu,i}(1-\delta) < S_{fu,i} * Y_{fu,i}$ 
        *[

```

```

        {'type': 'ineq', 'fun': lambda S_fu, i=i: M_fu[i] * (1 + delta) - S_fu[i] * Y_fu[i]}
        for i in range(4)
    ] # S_fu_i * Y_fu_i < M_fu_i(1+)
]
# 初始值
S_fu_initial = np.ones(4) * (S_E / 4) # 平均分配的初始值

# 求解优化问题
result = minimize(objective, S_fu_initial, constraints=constraints, method='SLSQP',
                  bounds=[(0, None)] * 4)

if result.success:
    return result.x
else:
    raise ValueError("Optimization did not converge")

df = pd.read_csv('../Data/data_fu.txt', sep='\s+')

Y_fu = df['Y(E2)'].values
P_fu = df['P(E2)'].values
C_fu = df['C(E2)'].values
M_fu = df['Y'].values
S_E = df['S'].sum()
delta = 1

S_fu_solution = surplus_unsold(Y_fu, P_fu, C_fu, M_fu, S_E, delta=0.3)
print("Solution for surplus unsold problem:", S_fu_solution)

S_fu_solution_discount = surplus_discount(Y_fu, P_fu, C_fu, M_fu, S_E, delta)
print("Solution for surplus with discount problem:", S_fu_solution_discount)

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

def expected_value(attr, t=7, num_samples=10000, up=0.10, bot=0.05):
    if np.isscalar(attr):
        Y_values = np.array([attr])
    else:
        Y_values = attr.values

    # 随机生成增长率
    r_values = np.random.uniform(0.05, 0.10, num_samples)

    # 计算每个 t 的期望值

```

```

expected_results = []

for i in range(0, t + 1):
    # 计算  $(1 + r)$  的  $i$  次方的期望值
    factors = (1 + r_values)**i
    E_factor = np.mean(factors)
    expected_results.append(E_factor)

# 计算每个  $t$  对应的  $E(P)$ 
results = np.array([Y_values * E for E in expected_results]).T

return results

def plwl(data, labels, title='粮食类作物(非豆类) 2023 - 2030 预期销售量变化(相对于 2023 年)',
        xlabel='时间', ylabel='预期销售量', fn='FutureCFd.png'):
    assert data.shape[0] == len(labels), "标签数量与数据行数不匹配"
    plt.rcParams['font.family'] = 'SimHei'
    plt.figure(figsize=(10, 6))

    for i, line in enumerate(data):
        plt.plot(line, label=labels[i])

    plt.legend()
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.tight_layout()
    plt.savefig(fn, bbox_inches="tight")
    plt.show()

def psb(array, label_array=None, colormap='Paired', title='粮食类作物年预期需求量变化',
        fn='FdStacked.png'):
    rows, cols = array.shape
    cmap = plt.get_cmap(colormap)
    colors = cmap(np.linspace(0, 1, rows))
    fig, ax = plt.subplots()
    bottom = np.zeros(cols)
    # 堆叠
    for i in range(rows):
        ax.bar(range(cols), array[i], bottom=bottom, color=colors[i], label=label_array[i])
        bottom += array[i]

    ax.legend(loc='upper left')
    ax.set_title(title)
    ax.set_xlabel('时间')
    ax.set_ylabel('值')
    plt.tight_layout()

```

```

plt.savefig(fn, bbox_inches="tight")
plt.show()

# fd
df_cfd = pd.read_csv('data_common_fd.txt', sep='\s+')
df_dou_fd = pd.read_csv('data_dou_abc.txt', sep='\s+')
df_rice = pd.read_csv('data_rice_fd.txt', sep='\s+')

# veg
df_dou_veg = pd.read_csv('data_dou_def.txt', sep='\s+')
df_cveg = pd.read_csv('data_common_veg.txt', sep='\s+')
df_rab = pd.read_csv('data_rab_veg.txt', sep='\s+')

# fu
df_fu = pd.read_csv('data_fu.txt', sep='\s+')

# fd 1, 2
df_2fd = df_cfd.iloc[:2]
# fd \neq 1, 2
df_cfd = df_cfd.iloc[2:].reset_index(drop=True) # c for common

df_fu1 = df_fu.iloc[-1]
df_cfu = df_fu.iloc[0:-1]

### 计算 fd 的变化
# 预期销售量
m_2fd = expected_value(df_2fd['Y'])
m_cfd = expected_value(df_cfd['Y'], up=0.05, bot=-0.05)
m_dfd = expected_value(df_dou_fd['Y'], up=0.05, bot=-0.05)
# 标签
l_2fd = df_2fd['作物名称'].values
l_cfd = df_cfd['作物名称'].values
l_dfd = df_dou_fd['作物名称'].values
# 价格, 稳定
p_2fd = df_2fd['P'].values
p_cfd = df_cfd['P'].values
p_dfd = df_dou_fd['P'].values
# 成本, 稳定
c_2fd = df_2fd['C'].values
c_cfd = df_cfd['C'].values
c_dfd = df_dou_fd['C'].values
# 预期亩产量的变动, 基本稳定
y_2fd = expected_value((df_2fd['Y(A)'] + df_2fd['Y(B)'] + df_2fd['Y(C)'])/3, up=0.1, bot=-0.1)
y_cfd = expected_value((df_cfd['Y(A)'] + df_cfd['Y(B)'] + df_cfd['Y(C)'])/3, up=0.1, bot=-0.1)
y_dfd = expected_value((df_dou_fd['Y(A)'] + df_dou_fd['Y(B)'] + df_dou_fd['Y(C)'])/3, up=0.1,
    bot=-0.1)

```

```

m_fd = np.concatenate((m_2fd, m_cfd, m_dfd), axis=0)
db_m_fd = m_fd - m_fd[:, [0]]# diff base
# diff_m_fd = m_fd[:, 1:] - m_fd[:, :-1]
l_fd = np.concatenate((l_2fd, l_cfd, l_dfd), axis=0)
p_fd = np.concatenate((p_2fd, p_cfd, p_dfd), axis=0)
y_fd = np.concatenate((y_2fd, y_cfd, y_dfd), axis=0)
c_fd = np.concatenate((c_2fd, c_cfd, c_dfd), axis=0)

plw1(db_m_fd, l_fd)
psb(db_m_fd, l_fd)

### 计算 Veg 的变化
# 预期销售量
m_cveg = expected_value(df_cveg['Y'], up=0.05, bot=-0.05)
m_rab = expected_value(df_rab['Y'], up=0.05, bot=-0.05)
m_dou_veg = expected_value(df_dou_veg['Y'], up=0.05, bot=-0.05)
# 标签
l_cveg = df_cveg['作物名称'].values
l_rab = df_rab['作物名称'].values
l_dou_veg = df_dou_veg['作物名称'].values
# 价格
p_cveg = expected_value((df_cveg['P(D1)'] + df_cveg['P(E1)'] + df_cveg['P(F2)'])/ 3, up=0.06,
    bot=0.04)
p_rab = expected_value(df_rab['P(D2)'], up=0.06, bot=0.04)
p_dou_veg = expected_value((df_cveg['P(D1)'] + df_cveg['P(E1)'] + df_cveg['P(F2)'])/ 3,
    up=0.06, bot=0.04)
# 成本, 稳定
c_cveg = (df_cveg['C(D1)'] + df_cveg['C(E1)'] + df_cveg['C(F2)'] / 3).values
c_rab = df_rab['C(D2)'].values
c_dou_veg = (df_dou_veg['C(D1)'] + df_dou_veg['C(E1)'] + df_dou_veg['C(F2)'] / 3).values
# 预期亩产量的变动, 基本稳定
y_cveg = expected_value((df_cveg['Y(D1)'] + df_cveg['Y(E1)'] + df_cveg['Y(F2)'])/3, up=0.1,
    bot=-0.1)
y_rab = expected_value(df_rab['Y(D2)'], up=0.1, bot=-0.1)
y_dou_veg = expected_value((df_dou_veg['Y(D1)'] + df_dou_veg['Y(E1)'] +
    df_dou_veg['Y(F2)'])/3, up=0.1, bot=-0.1)

m_veg = np.concatenate((m_cveg, m_rab, m_dou_veg), axis=0)
db_m_veg = m_veg - m_veg[:, [0]]# diff base
# diff_m_veg = m_veg[:, 1:] - m_veg[:, :-1]
l_veg = np.concatenate((l_cveg, l_rab, l_dou_veg), axis=0)
p_veg = np.concatenate((p_cveg, p_rab, p_dou_veg), axis=0)
y_veg = np.concatenate((y_cveg, y_rab, y_dou_veg), axis=0)
c_veg = np.concatenate((c_cveg, c_rab, c_dou_veg), axis=0)

plw1(db_m_veg, l_veg, title='蔬菜类作物 2023 - 2030 预期销售量变化(相对于 2023 年)',
    fn='FutureVeg.png')

```

```

psb(db_m_veg, l_veg, title='蔬菜类作物年预期需求量变化', fn='VegStacked.png')

### 计算 Fu 的变化
# 预期销售量
m_fu1 = expected_value(df_fu1['Y(E2)'], up=0.05, bot=-0.05)
m_cfu = expected_value(df_cfu['Y(E2)'], up=0.05, bot=-0.05)
# 标签
l_fu1 = np.array([df_fu1['作物名称']])
l_cfu = df_cfu['作物名称'].values
# 价格
p_fu1 = expected_value(df_fu1['P(E2)'], up=-0.06, bot=-0.04)
p_cfu = expected_value(df_cfu['P(E2)'], up=-0.01, bot=-0.05)
# 成本, 稳定
c_fu1 = np.array([df_fu1['C(E2)']])
c_cfu = df_cfu['C(E2)'].values
# 预期亩产量的变动, 基本稳定
y_fu1 = expected_value(df_fu1['Y(E2)'], up=0.1, bot=-0.1)
y_cfu = expected_value(df_cfu['Y(E2)'], up=0.1, bot=-0.1)

m_fu = np.concatenate((m_fu1, m_cfu), axis=0)
db_m_fu = m_fu - m_fu[:, [0]] # diff base
# diff_m_veg = m_veg[:, 1:] - m_veg[:, :-1]
l_fu = np.concatenate((l_fu1, l_cfu), axis=0)
p_fu = np.concatenate((p_fu1, p_cfu), axis=0)
y_fu = np.concatenate((y_fu1, y_cfu), axis=0)
c_fu = np.concatenate((c_fu1, c_cfu), axis=0)

plwl(db_m_fu, l_fu, title='食用菌作物 2023 - 2030 预期销售量变化(相对于 2023 年)',
      fn='FutureFu.png')
psb(db_m_fu, l_fu, title='食用菌作物年预期需求量变化', fn='FuStacked.png')

```

4.8 随机模拟求种植面积

```

import numpy as np

def monte_carlo_generate(I, T, S, beta, S_min, S_max, Q_init, max_iterations=1000):
    solutions = []

    for _ in range(max_iterations):
        # Initialize Q with the given Q_init for t=1
        Q = np.zeros((I, T, S), dtype=int)
        Q[:, 0, :] = Q_init # Set the first slice of Q based on Q_init

        # Randomly initialize the remaining elements of Q
        for i in range(I):

```

```

    for t in range(1, T):
        for s in range(S):
            Q[i, t, s] = np.random.choice([0, 1])

# Check the constraints
valid = True

# Constraint 1: Sum over t, t+2 should be at least 1
for i in range(I):
    for t in range(T-2): # Ensure no out-of-bounds access
        for s in range(S):
            if np.sum(Q[i, t:t+3, s]) < 1:
                valid = False
                break
    if not valid:
        break

# Constraint 2: Sum over t, t+1 should be at most 1
for i in range(I):
    for t in range(T-1): # Ensure no out-of-bounds access
        for s in range(S):
            if np.sum(Q[i, t:t+2, s]) > 1:
                valid = False
                break
    if not valid:
        break

# Constraint 3: Weighted sum must be within bounds
for i in range(I):
    for t in range(1, T):
        weighted_sum = np.sum(Q[i, t, :] * beta)
        if not (S_min[i] <= weighted_sum <= S_max[i]):
            valid = False
            break
    if not valid:
        break

    if valid:
        solutions.append(Q.copy())

return solutions

# Example usage:
I, T, S = 5, 8, 3 # Dimensions
beta = np.array([0.5, 0.3, 0.2]) # Weight vector
S_min = np.array([1, 2, 1, 3, 2]) # Min sum constraints for each i
S_max = np.array([3, 3, 2, 4, 3]) # Max sum constraints for each i

```

```

Q_init = np.random.choice([0, 1], size=(I, S)) # Initial Q for t=1

solutions = monte_carlo_generate(I, T, S, beta, S_min, S_max, Q_init)
print(f"Found {len(solutions)} solutions.")

```

4.9 采取遗传算法求解种植策略

```

import random
import pandas as pd
import numpy as np
from deap import base, creator, tools, algorithms

df = pd.read_csv('../Data/data_rab_veg.txt', sep='\s+')

Y = df['Y(D2)'].values
P = df['P(D2)'].values
C = df['C(D2)'].values
M = df['Y'].values
S_D = df['S'].sum()
delta = 0.1

# 创建最小化目标
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()
toolbox.register("attr_float", random.uniform, 0, S_D)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, n=3)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# 定义第一个问题的适应度函数
def fitness_1(individual):
    profit = sum(individual[i] * (Y[i] * P[i] - C[i]) for i in range(3))
    # 约束条件: M[i](1 - delta) < individual[i] * Y[i] < M[i]
    for i in range(3):
        if not (M[i] * (1 - delta) < individual[i] * Y[i] < M[i]):
            return -1e6, # 惩罚违反约束的个体
    if sum(individual) >= S_D:
        return -1e6, # 总和超过 S_D 的情况也要惩罚
    return profit,

# 定义第二个问题的适应度函数
def fitness_2(individual):
    profit_full = sum(min(individual[i] * Y[i], M[i]) * (P[i] - C[i] / Y[i]) for i in range(3))
    profit_discount = sum(max((individual[i] * Y[i] - M[i]), 0) * (0.5 * P[i] - C[i] / Y[i])

```

```

        for i in range(3))
    total_profit = profit_full + profit_discount
    # 约束条件:  $M[i](1 - \delta) < \text{individual}[i] * Y[i] < M[i]$ 
    for i in range(3):
        if not ( $M[i] * (1 - \delta) < \text{individual}[i] * Y[i] < M[i] * (1 + \delta)$ ):
            return -1e6,
    if sum(individual) >= S_D:
        return -1e6,
    return total_profit,

# 注册遗传算法工具
toolbox.register("evaluate", fitness_1) # 选择 fitness_1 或 fitness_2
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

# 遗传算法主循环
def main():
    pop = toolbox.population(n=100)
    hof = tools.HallOfFame(1)
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("avg", lambda x: sum(x) / len(x))
    stats.register("max", max)

    algorithms.eaSimple(pop, toolbox, cxpb=0.7, mutpb=0.2, ngen=50, stats=stats,
                        halloffame=hof, verbose=True)

    return hof[0] # 返回最优个体

if __name__ == "__main__":
    best_ind = main()
    print("最优解决方案:", best_ind)

```